



Large Language Models (LLMs) *- Data and Models*

Rowel Atienza, PhD

University of the Philippines

github.com/roatienza

2025

<https://stanford-cs324.github.io/winter2022>

Data

Data for LLMs

- Must be diverse
- Sources : Web
 - Common Crawl – free and open, 320TB
- Other sources: Private datasets

Representation

- Web data – mostly from young people and developed countries
- GPT2 – trained on Reddit and WebText; most contributors are men
- Wikipedia – at most 15% contributors are women

WebText

- Diverse but high quality
- News, wikipedia, fiction
- 40GB of text

OpenWebText - OpenAI

- Extracted all the URLs from the Reddit submissions dataset.
- Used Facebook's fastText to filter out non-English.
- Removed near duplicates.
- End result is 38 GB of text.
- Small toxicity content

Colossal Clean Crawled Corpus (C4)

- Started with April 2019 snapshot of Common Crawl (1.4 trillion tokens)
- Removed “bad words”
- Removed code (“{”)
- langdetect to filter out non-English text
- Resulted in 806 GB of text (156 billion tokens)

GPT3 Dataset

- Downloaded 41 shards of Common Crawl (2016-2019).
- 13-gram deduplication
- No benchmark datasets
- Expanded data sources (WebText2, Books1, Books2, Wikipedia).

The Pile – Eleuther AI

- A dataset for language modeling, where the key idea is to source it from smaller high-quality sources (academic + professional sources).
- 825 GB English text
- 22 high-quality datasets

Component	Raw Size	Weight	Epochs	Effective Size	Mean Document Size
Pile-CC	227.12 GiB	18.11%	1.0	227.12 GiB	4.33 KiB
PubMed Central	90.27 GiB	14.40%	2.0	180.55 GiB	30.55 KiB
Books3 [†]	100.96 GiB	12.07%	1.5	151.44 GiB	538.36 KiB
OpenWebText2	62.77 GiB	10.01%	2.0	125.54 GiB	3.85 KiB
ArXiv	56.21 GiB	8.96%	2.0	112.42 GiB	46.61 KiB
Github	95.16 GiB	7.59%	1.0	95.16 GiB	5.25 KiB
FreeLaw	51.15 GiB	6.12%	1.5	76.73 GiB	15.06 KiB
Stack Exchange	32.20 GiB	5.13%	2.0	64.39 GiB	2.16 KiB
USPTO Backgrounds	22.90 GiB	3.65%	2.0	45.81 GiB	4.08 KiB
PubMed Abstracts	19.26 GiB	3.07%	2.0	38.53 GiB	1.30 KiB
Gutenberg (PG-19) [†]	10.88 GiB	2.17%	2.5	27.19 GiB	398.73 KiB
OpenSubtitles [†]	12.98 GiB	1.55%	1.5	19.47 GiB	30.48 KiB
Wikipedia (en) [†]	6.38 GiB	1.53%	3.0	19.13 GiB	1.11 KiB
DM Mathematics [†]	7.75 GiB	1.24%	2.0	15.49 GiB	8.00 KiB
Ubuntu IRC	5.52 GiB	0.88%	2.0	11.03 GiB	545.48 KiB
BookCorpus2	6.30 GiB	0.75%	1.5	9.45 GiB	369.87 KiB
EuroParl [†]	4.59 GiB	0.73%	2.0	9.17 GiB	68.87 KiB
HackerNews	3.90 GiB	0.62%	2.0	7.80 GiB	4.92 KiB
YoutubeSubtitles	3.73 GiB	0.60%	2.0	7.47 GiB	22.55 KiB
PhilPapers	2.38 GiB	0.38%	2.0	4.76 GiB	73.37 KiB
NIH ExPorter	1.89 GiB	0.30%	2.0	3.79 GiB	2.11 KiB
Enron Emails [†]	0.88 GiB	0.14%	2.0	1.76 GiB	1.78 KiB
The Pile	825.18 GiB			1254.20 GiB	5.91 KiB

Modeling

Language Model

- Probability distribution over sequences of tokens
- The LM:

$$p(x_1, x_2, \dots, x_L) = p \in [0,1]$$

- LM using **Prompt** to **Completion**

$$p(x_1, x_2, \dots, x_L) = \\ p(x_1, x_2, \dots, x_P) p(x_{P+1} | x_1, x_2 \cdots x_P) \cdots p(x_L | x_1, x_2 \cdots x_{L-1})$$

Where $x_1, x_2, \dots, x_L \in \mathcal{V}$. L is the sequence length. Assumption is there an existing a vocabulary $\mathcal{V}$

Language Model

Goal: Learn $p(x_1, x_2, \dots, x_L)$ from data (eg. The Pile)

Tokenization

https://huggingface.co/docs/transformers/en/tokenizer_summary

Tokenization

Tokenization: how a string is split into tokens.

Tokenization

1. *Tokens*: Split the text into words or subwords.
2. *Token IDs*: Convert words and subwords into IDs or integers using a lookup table.

Recall: Model Architecture

Model: Estimates $p(x_1, x_2, \dots, x_L)$ from data represented as tokens. $x_1, x_2, \dots, x_L$ are the tokens.

Tokenization

- Language comes as a string or text
 - *Language*: “the cat sat on the mat”
- A tokenizer converts a string into tokens and token IDs
 - *Tokens*: “the cat sat on the mat” → [the cat sat on the mat]
 - *Token IDs*: [0, 1, 2, 3, 0, 4]
- Easiest :

```
[>>> text = "the cat sat on the mat"  
[>>> text.split(" ")  
['the', 'cat', 'sat', 'on', 'the', 'mat']
```

Tokenization

- However, in some languages (and some words in English), spaces do not separate words

What is a good tokenizer?

Good Tokenizer

- **Coverage & Vocabulary**
- Handles the full range of input (languages, symbols, code, numbers, special characters)
- Balances vocabulary size — too small means long sequences; too large means rare/unseen tokens
- Gracefully handles out-of-vocabulary words (e.g., via subword fallback)

Good Tokenizer

- **Efficiency**
- Produces short token sequences (fewer tokens = lower compute cost and longer effective context)
- Common words/phrases map to single tokens; rare ones decompose sensibly

Good Tokenizer

- **Consistency**
- Same input always produces the same output (deterministic)
- Reversible — tokenization is lossless and the original text can be reconstructed exactly

Good Tokenizer

- **Linguistic Awareness**
- Respects meaningful boundaries (morphemes, words, punctuation) where possible
- Handles whitespace, capitalization, and casing predictably
- Doesn't split semantically atomic units unnecessarily (e.g., "don't" → don + 't is okay; splitting NASA into N + A + S + A is wasteful)

Good Tokenizer

- **Cross-lingual Fairness**
- Doesn't over-penalize non-English languages with disproportionately long token sequences
- Allocates vocabulary fairly across scripts (Latin, CJK, Arabic, Cyrillic, etc.)

Good Tokenizer

- **Domain Adaptability**
- Works well for the target domain — code tokenizers should preserve indentation and syntax; medical tokenizers should handle clinical terms efficiently

Good Tokenizer

- **Speed**
- Fast enough to not be a bottleneck during training or inference, especially at scale

Good Tokenizer

- **Robustness**
- Handles edge cases: empty strings, very long words, Unicode, emojis, mixed-script inputs
- Doesn't fail or degrade badly on noisy/malformed input

Tokenizers

Byte Pair Encoding (BPE) [Sennrich et al, 2015]

- Learning the tokenizer. Intuition: start with each character as its own token and combine tokens that co-occur a lot.
- Input: a training corpus (sequence of characters).
- Initialize the vocabulary $x_1, x_2, \dots, x_L \in \mathcal{V}$
- While we want to still grow $\mathcal{V}$:
- Find the pair of elements $x_i, x_j \in \mathcal{V}$ that co-occur the most number of times.
- Replace all occurrences of x_i, x_j with a new symbol x_{ij} .
- Add x_{ij} to $\mathcal{V}$.

Example

- Data: $[t\ h\ e\ _\ c\ a\ t], [t\ h\ e\ _\ c\ a\ p], [t\ h\ e\ _\ b\ a\ t]$
- New tokens:
 - $th - 3 \times$
 - $the - 3 \times$
 - $ca - 2 \times$
 - $at - 2 \times$
- Updated $\mathcal{V}$: $t, h, e, c, a, t, p, b, th, the, ca, at$
- $Tokenize([t\ h\ e\ _\ c\ a\ t]) = [the\ _\ ca\ t]$

Unicode

- 144,697 of Unicode characters.
- Solution: Run on byte equivalent of characters

SentencePiece (Unigram Model) [Kudo 2018]

$$p(x_{1:L}) = \prod_{(i,j) \in \mathcal{T}} p(x_{i:j})$$

- Training: $[ababc]$
- Vocabulary: $\mathcal{V} = \{ab, c\}$
- Likelihood: $p(x_{1:5}) = \frac{2}{3} \times \frac{2}{3} \times \frac{1}{3} = \frac{4}{9}$

SentencePiece (Unigram Model) [Kudo 2018]

- Fully reversible (lossless) – tokenized strings can be detokenized w/o ambiguity.
 - `detokenize(tokenize('I love New York.')) == 'I love New York.'`
- Uses both BPE and unigram for tokenization
- Uses Unicode characters, including whitespace, and using a consistent encoding and decoding scheme to preserve all the information needed to reproduce the original text
- Fast, self-contained, language independent

<https://github.com/google/sentencepiece>

<https://medium.com/codex/sentencepiece-a-simple-and-language-independent-subword-tokenizer-and-detokenizer-for-neural-text-ffda431e704e>

SentencePiece Algorithm

1. Start with a “reasonably big” seed vocabulary $\mathcal{V}$.
 - $\mathcal{V}$: $[ababc] \rightarrow [abab\ aba\ ab\ a\ b\ c]$
 - Suffix array algorithm
2. Repeat:
 - a) Given $\mathcal{V}$, optimize $p(x)$ and $\mathcal{T}$ using the EM algorithm.
 - b) Compute $loss(x)$ for each token $x \in \mathcal{V}$ capturing how much the likelihood would be reduced if x were removed from $\mathcal{V}$.
 - c) Sort by loss and keep the top 80% tokens in $\mathcal{V}$.

EM

- Loss of all tokens:

$$\mathcal{L} = - \sum N_i \log p_i(x)$$

$N_i = \text{frequency of token } x_i$

$$p_i(x) = \frac{\text{frequency of } x_i}{\sum_j x_j}$$

EM

$$\mathcal{V}: [ababc] \rightarrow [abab\ aba\ ab\ a\ b\ c]$$

$$\sum_j x_j = 1 + 1 + 2 + 2 + 2 + 1 = 9$$

$p(abab) = p(aba) = p(c)$	$\frac{1}{9}$
$p(ab) = p(a) = p(b)$	$\frac{2}{9}$

EM

- Single characters are never removed to ensure that any unknown word can still be broken down into its base components
- The tokens that, when removed, cause the smallest change in total loss are pruned
 - $\mathcal{V}: [ababc] \rightarrow [abab\ aba\ ab\ a\ b\ c]$
 - $\mathcal{L} = -\sum N_i \log p_i(x) = -1\log\frac{1}{9} - 1\log\frac{1}{9} - 2\log\frac{2}{9} - 2\log\frac{2}{9} - 2\log\frac{2}{9} - 1\log\frac{1}{9} = 15.6162$

EM – abab or aba is removed

$$\mathcal{V}: [ababc] \rightarrow [aba \ ab \ a \ b \ c]$$
$$\sum_j x_j = 1 + 2 + 2 + 2 + 1 = 8$$

$p(aba) = p(c)$	$\frac{1}{8}$
$p(ab) = p(a) = p(b)$	$\frac{2}{8}$

EM – abab or aba is removed

- $\mathcal{V}: [ababc] \rightarrow [aba \ ab \ a \ b \ c]$
- $\mathcal{L} = -\sum N_i \log p_i(x) = -1\log\frac{1}{8} - 2\log\frac{2}{8} - 2\log\frac{2}{8} - 2\log\frac{2}{8} - 1\log\frac{1}{8} = 13.4765$

EM – ab is removed

$\mathcal{V}: [ababc] \rightarrow [abab\ aba\ a\ b\ c]$

$$\sum_j x_j = 1 + 1 + 2 + 2 + 1 = 7$$

$p(abab) = p(aba) = p(c)$	$\frac{1}{7}$
$p(a) = p(b)$	$\frac{2}{7}$

EM – ab is removed

- $\mathcal{V}: [ababc] \rightarrow [abab\ aba\ a\ b\ c]$
- $\mathcal{L} = -\sum N_i \log p_i(x) = -1\log\frac{1}{7} - 1\log\frac{1}{7} - 2\log\frac{2}{7} - 2\log\frac{2}{7} - 1\log\frac{1}{7} = 10.8487$

SentencePiece vs BPE

- GPT-2 and GPT-3 used BPE, vocabulary size of 50K
- Jurassic used SentencePiece with vocabulary size of 256K
- Impact:
 - Given the same string, Jurassic requires 28% fewer tokens than GPT-3, so it is 1.4x faster
 - Both Jurassic and GPT-3 use the same context size (2048), so one can feed in 39% more text into the prompt.

Example

- GPT-3 BPE (9 tokens):
 - [Ab, raham, _Lincoln, _lived, _at, _the, _White, _House, .]
- Jurassic SentencePiece (4 tokens):
 - [Abraham_Lincoln, _lived, _at_the_White_House, .]

LM Types

LM Types

- Encoder Only – BERTs
- Decoder Only – GPTs
- Encoder-Decoder – BART, T5

Embeddings

- Embedding: $\phi: \mathcal{V}^L \rightarrow \mathbb{R}^{d \times L}$
 - Tokens to features
- Embedding: $x_{1:L} = [x_1, x_2, \dots, x_L] \xrightarrow{\phi} [\phi(x_1), \phi(x_2), \dots, \phi(x_L)]$
- Example:

$$\begin{aligned} x_{1:3} = [ab, ab, c] &\xrightarrow{\phi} [\phi(ab), \phi(ab), \phi(c)] = \left[\begin{bmatrix} 1. \\ -1. \\ 0. \end{bmatrix}, \begin{bmatrix} 1 \\ -1 \\ 0. \end{bmatrix}, \begin{bmatrix} 0. \\ 1. \\ 1. \end{bmatrix} \right] \\ &= \phi(x_{1:3}) \end{aligned}$$

Static Embeddings

- Static Embeddings (e.g., Word2Vec, GloVe, fastText) assign a fixed vector to each word, regardless of the context in which it appears.
 - Example: The word "bank" will have the same vector whether it means a financial institution or the side of a river.
 - Limitation: They cannot handle polysemy (words with multiple meanings) because the meaning of a word is not context-dependent.

Contextual Embeddings

- Contextual embedding is a more advanced form of word representation where the vector for a word is dynamically generated based on the surrounding words in a sentence.
- The model processes the entire sentence using self-attention mechanisms (in transformers) to understand how words relate to each other.

Contextual Embeddings

- The final vector for a word depends on its *context* (i.e., the words before and after it).

Example:

In the sentence: "I deposited money in the bank."

→ The word "bank" is represented as a financial institution.

In: "I sat by the bank of the river."

→ The word "bank" is represented as a riverbank.

- The vector for "bank" is different in each case.

Encoder Only (BERT, RoBERTa)

- Can produce contextual embedding: $x_{1:L} \xrightarrow{\phi} \phi(x_{1:L})$
- Application 1: Sentiment classification
 - [[CLS],the,movie,was,great]⇒positive.
- Application 2: Natural language inference
 - [[CLS],all,animals,breathe,[SEP],cats,breathe]⇒entailment.
 - Text ⇒ Hypothesis
- Pro: bidirectional dependency
- Con: Cannot generate text
- Loss: Masked Language Modelling

Decoder Only (GPTs) – Autoregressive Models

- Can produce contextual embedding of the prompt: $x_{1:P} \xrightarrow{\phi} \phi(x_{1:P})$ and the distribution over the next tokens $p(x_{P+1:L})$ or to the completion.

$$x_{1:P} \Rightarrow \phi(x_{1:P})p(x_{P+1:L}|\phi(x_{1:P}))$$

- Application: Text Generation
 - `[[CLS],the,movie,was]⇒great`
- Pro: Can naturally generate text completions.
- Con: Unidirectional dependency (causal)
- Loss: Maximum Likelihood

Encoder-Decoder (T5, BART)

- Can produce contextual embedding: $x_{1:L} \xrightarrow{\phi} \phi(x_{1:L})$ and generate new output : $y_{1:L}$.

$$x_{1:L} \Rightarrow \phi(x_{1:L})p(y_{1:L}|\phi(x_{1:L}))$$

- Application: Table to text conversion
- Pro: bidirectional dependency.
- Con: can generate new outputs
- Loss: Ad-hoc training objectives

General Algorithm for LMs

1. Given an input string: x
2. $y_{tok} = Tokenize(x)$
3. $y_{emb} = Embed(y_{tok})$ or $y_{ctx} = ContextEmbed(y_{emb})$
4. $y_{seq} = SequenceModel(y_{emb})$ or $SequenceModel(y_{ctx})$

General Algorithm for LMs

1. Given an input string: x
2. $Tokenize(x)$ - Tokenizer (eg sentencepiece, BPE, wordpiece)
3. $Embed(y_{tok})$ - `nn.Embedding()`
4. $ContextEmbed(y_{emb})$ - `nn.Transformer()`
5. $SequenceModel(y_{emb})$ or $SequenceModel(y_{ctx})$

End